第二章 进程与线程

2022年9月25日 星期日 18:13

进程【Process】

★ 进程常考的知识点

关于交换数据	1.进程之间不能直接交换数据，但可利用OS提供的共享文件(如管道)，信息传递系统，共享存储区进行通信 2.全局变量是针对一个进程而言的，不同的进程拥有不同的代码段和数据段，不能通过全局变量来交换进程之间的数据 3.临界资源一次只能为一个进程所用
关于进程的特性	1.进程失去封闭性（进程执行的结果只取决于进程本身）后，不同速度下的执行结果不同 2.进程被建立后，随着进程运行的正常或不正常结束而撤销 3.进程不是一个完整的程序 进程是动态的，程序是静态的 进程是暂时的，程序是永久的 进程至少由代码，数据，pcb组成，程序仅需代码和数据 4.一个进程是程序在一个数据集上的一次运行过程；运行于不同的数据集，会形成不同的进程
关于进程与进程的关系	1.进程与进程之间是完全隔离的，线程与线程之间是没有隔离的，但并不意味着进程之间是毫无关联的 • 如一个进程的状态变化可能会引起另一个进程的状态变化 • 如打印机进程结束后可能会改变另一个等待打印机的进程
关于各种状态	1.死锁的进程是处于阻塞态的，不可能所有的进程都处于就绪态，但是可以所有的进程都处于阻塞态(死锁的时候) 2.就绪队列进程的多少与处理器的效率无关，只有当就绪队列为空时，CPU进入等待态，cpu效率下降
关于进程里的信息	1.C语言的程序在使用内存时有3个段

	存放什么?	举例
正文段	存二进制代码，常量	全局赋值变量，常量值123
数据堆段	存动态分配的存储区	用malloc动态分配的存储区
数据栈段	存临时使用的变量	未赋值的局部变量，函数调用实参传递值

进程的概念与特征

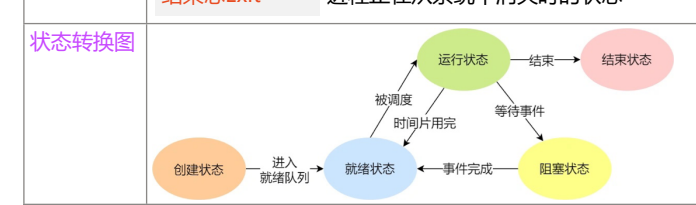
定义	• 进程是程序的一次执行过程 • 进程是一个程序及其数据在处理机上顺序执行时所发生的活动 • 进程是具有独立功能的程序在一个数据集合上运行的过程，是系统进行资源分配和调度的一个独立单位
特征	• 动态性 • 并发性 • 独立性 • 异步性
为什么要引入进程?	• 为了使多道程序并发执行，提高资源利用率和系统吞吐量
其他考点	• 进程实体/进程映像 = PCB + 程序段 + 相关数据段 • 进程映像是静态的，进程是动态的，程序是静态的 • 进程控制块(PCB, Process Control Block)是进程的唯一标志 • 系统资源 = 处理机，存储器和其他设备服务于某个进程的"时间"【这里的系统资源类似于处理机的时间片】

进程的状态与转换

五种状态

运行态Running	该时刻进程占用CPU
就绪态Ready	进程获得了除处理机外的一切所需资源，一旦得到处理机，就可以立即运行
阻塞态Blocked	该进程正在等待某一事件发生而暂停运行，即使给他CPU控制权，它也无法运行
创建态New	进程正在被创建时的状态
结束态Exit	进程正在从系统中消失时的状态

状态转换图



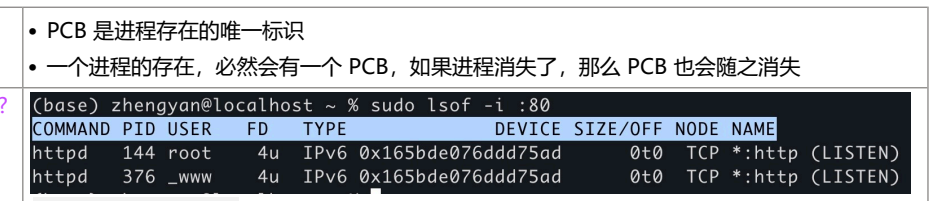
状态的转换

就绪态-->运行态	1. 进程被调度，获得处理机资源（分派处理机时间片）
运行态-->就绪态	1. 时间片用完后，不得不让出处理机 2. 可剥夺的OS中，当有更高优先级的进程就绪时，调度程序将正在执行的进程转换为就绪态
运行态-->阻塞态	1. 进程请求某一资源（如外设）的使用和分配时 2. 等待某一事件的发送时（如I/O操作的完成） 进程以系统调用的方式请求OS提供服务，这个过程系统从用户态转换为核心态 该过程是主动行为
阻塞态-->就绪态	1. I/O操作结束或中断结束时 该过程是被动行为，需要其他相关进程的协助

进程的控制结构/数据结构

- 进程的控制结构 = PCB + 程序段 + 数据段

1、PCB	• PCB 是进程存在的唯一标识 • 一个进程的存在，必然会有一个 PCB，如果进程消失了，那么 PCB 也会随之消失
PCB包含什么信息?	进程描述信息：• 进程标识符PID：标识各个进程，每个进程都有一个并且唯一的标识符 • 用户标识符UID：进程归属的用户，用户标识符主要为共享和保护服务 进程控制和管理信息：• 进程当前状态，如 new、ready、running、waiting 或 blocked 等 • 进程优先级：进程抢占 CPU 时的优先级 资源分配清单：• 有关内存地址空间或虚拟地址空间的信息 • 所打开文件的列表和所使用的 I/O 设备信息 CPU 相关信息：• CPU 中各个寄存器的值 • 当进程被切换时，CPU 的状态信息都会被保存在相应的 PCB 中以便进程重新执行时，能从断点处继续执行
如何组织PCB?	链接方式：• 把统一状态的PCB链成一个队列，不同状态对应不同的队列 • 也可把处于阻塞态的进程的PCB，根据阻塞原因，排成多个阻塞队列 索引方式：• 将统一状态的进程组织在一个索引表中 • 如就绪索引表，阻塞索引表
2、程序段	• 能被进程调度程序调度到CPU执行的程序代码段 • 程序可以被多个进程共享，即多个进程可以运行同一个程序
3、数据段	• 可以是进程对应的程序加工处理的原始数据 • 可以是程序执行时产生的中间或最终结果



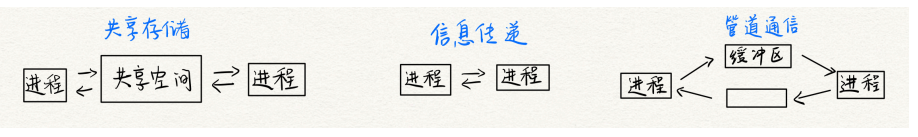
```
(base) zhengyan@localhost ~ % sudo lsof -i :80
COMMAND PID USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
httpd   144 root    4u  IPv6 0x165bde076ddd75ad      0t0  TCP *:http (LISTEN)
httpd   376 _www    4u  IPv6 0x165bde076ddd75ad      0t0  TCP *:http (LISTEN)
```

进程的控制（也就是进程的创建、终止、阻塞、唤醒的过程）

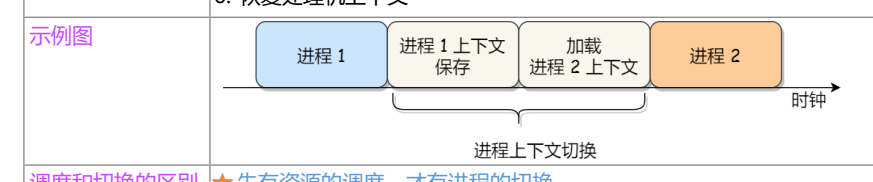
	定义和过程	对应事件
创建	• 允许一个进程创建另一个进程 • 允许子进程继承父进程所拥有的资源 • 创建进程的过程如下: • 申请一个空白的 PCB，并向 PCB 中填写一些控制和管理进程的信息，比如进程的唯一标识等； • 为该进程分配运行时所必需的资源，比如内存资源； • 将 PCB 插入到就绪队列，等待被调度运行；	1. 终端登陆系统，作业调度 2. 系统提供服务 3. 用户程序的应用 设备分配不需要创建进程
终止	• 有 3 种终止方式：正常结束、异常结束以及外界干预 • 当子进程被终止时，其在父进程处继承的资源应当还给父进程 • 当父进程被终止时，该父进程的子进程就变为孤儿进程 • 终止进程的过程如下: • 查找需要终止的进程的 PCB； • 如果处于执行状态，则立即终止该进程的执行，然后将 CPU 资源分配给其他进程； • 如果其还有子进程，则应将该进程的子进程交给 1 号进程接管； • 将该进程所拥有的全部资源都归还给操作系统； • 将其从 PCB 所在队列中删除；	1. 正常结束 2. 异常结束 3. 外界干预
阻塞	• 当进程需要等待某一事件完成时，它可以调用阻塞语句把自己阻塞等待 • 一旦被阻塞等待，只能由另一个进程唤醒 • 阻塞进程的过程如下: • 找到将要被阻塞进程标识号对应的 PCB； • 如果该进程为运行状态，则保护其现场，将其状态转为阻塞状态，停止运行； • 将该 PCB 插入到阻塞队列中去；	1. 请求系统资源失败 2. 等待某种操作的完成 3. 新数据尚未到达或无新任务可做
唤醒	• 进程由「运行」转变为「阻塞」状态是由于进程必须等待某一事件的完成 • 处于阻塞状态的进程是绝对不可能叫醒自己 • 如果某进程正在等待 I/O 事件，需由别的进程发消息给它 • 只有当该进程所期待的事件出现时，才由发现者进程用唤醒语句叫醒它 • 唤醒进程的过程如下: • 在该事件的阻塞队列中找到相应进程的 PCB； • 将其从阻塞队列中移出，并置其状态为就绪状态； • 把该 PCB 插入到就绪队列中，等待调度程序调度；	1. 释放该I/O设备的进程 2. 提供数据的进程

进程的通信

定义	进程通信是指进程之间的信息交换
低级通信方法	PV操作
高级通信方法	1、共享存储：• 如共享内存，共享文件系统 2、信息传递：• 需要发送消息和接受信息两个原语 3、管道通信：• 需要满足的能力 = 互斥 + 同步 + 确定对方的存在 • 缓冲区只允许一边写入，另一边读出，所以管道只能采用半双工通信 • 管道是一种文件，所以两个进程可以通过文件系统交换数据 示例图



进程的上下文切换

上下文	上下文是指某一时刻CPU寄存器和PC的内容
进程上下文切换	上下文切换是指一个进程切换到另一个进程的过程
上下文切换的实质	上下文切换实质是处理机从一个进程的运行转到另一个进程上运行，这个过程中，进程的运行环境产生了实质性的变化
上下文切换的消耗	有些处理器提供多个寄存器组，上下文切换只需要简单改变当前寄存器组的指针
上下文切换的场景	1. 某个进程时间片耗尽时 2. 进程在系统资源不足时，要等到资源满足后才可以运行 3. 进程通过sleep将自己主动挂起 4. 有优先级更高的基础运行时 5. 发送硬件中断时
上下文切换的流程	1. 挂起一个进程，保存CPU上下文，包括PC和其他寄存器 2. 更新PCB信息 3. 把进程的PCB移到相应的队列（如就绪队列，阻塞队列） 4. 加载另一个进程执行，更新其PCB 5. 跳转到新进程PCB中的PC所指向的位置执行 6. 恢复处理机上下文
示例图	
调度和切换的区别	★ 先有资源的调度，才有进程的切换 调度：• 决定资源分配给哪个进程的行为；是一种决策行为 切换：• 实际分配的行为；是一种执行行为

线程【Thread】

★ 线程常考的知识点

关于线程的特征	1.线程是处理机调度的基本单位，可以独立执行程序 2.没有相同的进程，但是调用在不同进程中的线程可以相同
关于线程可共享的东西	1.线程没有自己独立的地址空间，每个线程只拥有自己的独立的栈空间 2.线程之间可以共享进程的资源(如代码段，堆空间，数据段，打开的文件) 3.线程之间不能共享虚拟地址空间，线程之间不能共享栈指针
线程的并行与并发	1.同一进程或不同进程内的线程都可以并发执行 2.进程之间可以并发但不可以并行，同一进程之间的线程可以并行
其他	1.键盘驱动程序不能体现多线程系统的特点，因为计算机一般只有一个键盘

★ 线程与进程的比较

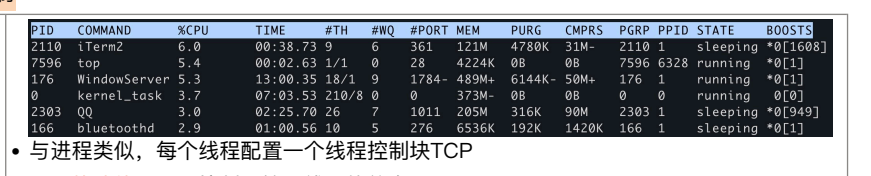
	进程	线程
式什么的单位?	是资源分配的基本单位	是调度的基本单位
不能共享什么?	不能共享虚拟地址空间	不能共享栈指针
可以共享什么?	拥有一个完整的资源平台 每个进程都有独立的地址空间和资源 除了共享全局变量，不允许其他进程访问	某进程中的线程对其他进程不可见 线程共享进程的地址空间和资源，线程自己没有独立的地址空间 只独享必不可少的资源，如寄存器和栈
进程切换会引起什么?	不同进程的线程的切换会引起进程切换	同一进程的线程的切换不会引起进程切换
创建,终止,切换时间	慢	快
数据交换效率	低	各线程间共享内存和文件资源，数据交换效率高

线程的概念与特征

定义	• 线程是进程当中的一条执行流程 • 线程之间可以并发运行且共享相同的地址空间 • 线程自己不拥有系统资源，但可与同属一个进程的程序共享进程所有的资源 • 线程同样具有三种基本状态：运行态+阻塞态+就绪态
为什么要引入线程?	为了减少程序在并发执行过程中所付出的时空开销，提高操作系统的并发能力
为什么线程有利于提高系统并发性?	线程可以导致平均每次切换所需开销变少，使得更多线程参与并发

线程的组成和控制

线程的组成	• 与进程类似，每个线程配置一个线程控制块TCP • TCP的功能：记录控制和管理线程的信息 • TCP的组成：线程标识符+一组寄存器+线程运行状态+优先级+线程专有存储区+堆栈指针
控制线程	• 创建线程 • 终止线程



```
PID   COMMAND      %CPU TIME     #TH    #WQ  #PORT MEM   PURG   CMPRS PGRP PPID STATE    BOOSTS
2110  iTerm2       6.0  00:38.73 9      6    361   121M  4780K  31M-  2110 1    sleeping *0[1608]
7596  top          5.4  00:02.63 1/1    0    28    4224K 0B     0B    7596 6328 running  *0[1]
176   WindowServer 5.3  13:00.35 18/1   9    1784- 489M+ 6144K- 50M+  176  1    running  *0[1]
0     kernel_task  3.7  07:03.53 210/8  0    0     373M- 0B     0B    0    0    running   0[0]
2303  QQ           3.0  02:25.70 26     7    1011  205M  316K   90M   2303 1    sleeping *0[949]
166   bluetoothd   2.9  01:00.56 10     5    276   6536K 192K   1420K 166  1    sleeping *0[1]
```

线程的分类

- 线程的实现分为两类 = 用户级线程(User-Level Thread, ULT) + 内核级线程(Kernel-Level Thread, KLT)

	ULT	KLT	组合模式
定义	由用户级线程库函数来完成整个线程的管理和调度【库函数负责】	线程对应的TCB放在OS里，线程的管理和调度由OS负责【OS负责】	内核支持的用户线程
模型	多对一模型（多个用户对应一个线程）	一对一模型（一个用户对应一个线程）	多对多模型
优点	• TCB由用户级线程库函数维护，可用于不支持线程技术的OS • 无需用户态和内核态的切换，速度特别快	• 某个内核线程发起系统调用被阻塞，不会影响到其他内核线程的运行 • 内核能同时调度同一进程中的多个线程并行执行	
缺点	• 一个线程发起了系统调用而阻塞，那进程所包含的用户线程都不能执行了 • 在多线程执行时，每个线程得到的时间片少，执行慢 • 线程与线程之间不用内核切换，但是跨进程的话就需要内核参与	• 同一进程的线程切换，需要从用户态转到核心态，系统开销大	

多线程模型

- 由于ULT和KLT连接方式的不同，形成了3种模式的多线程模型

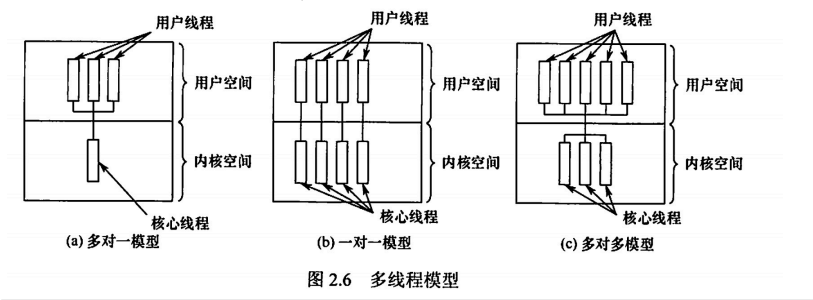


图 2.6 多线程模型

	多对一模型	一对一模型	多对多模型
定义	多个ULT映射到一个KLT	每个ULT映射到一个KLT	n个ULT映射到m个KLT，n ≥ m
优点	• 线程管理在用户空间进行，效率高	• 一个线程被阻塞，运行调度另一个线程运行，并发能力强	• 克服了多对一模型的并发度不高的缺点 • 克服了一对一模型的一个用户进程占用太多内核线程而开销太大的缺点
缺点	• 如果一个线程阻塞，其他进程都会被阻塞 • 任何时刻，只有一个线程能访问内核 • 多个线程不能同时在多个处理机上运行	• 每创建一个用户线程，就要创建一个对应的内核线程，开销大	

处理机调度

调度常考的知识点

1.作业是用户提交的，以用户任务为单位

2.进程是系统自动生成的，以操作系统控制为单位

基本概念

1.调度是处理机进行分配，即从就绪队列中按一定算法(公平，高效的原则)选择一个进程并将处理机分配给他运行，以实现进程并发地执行

2.调度是多道程序OS的基础；调度是OS设计的核心问题

调度的层次分类

层次分类模型图

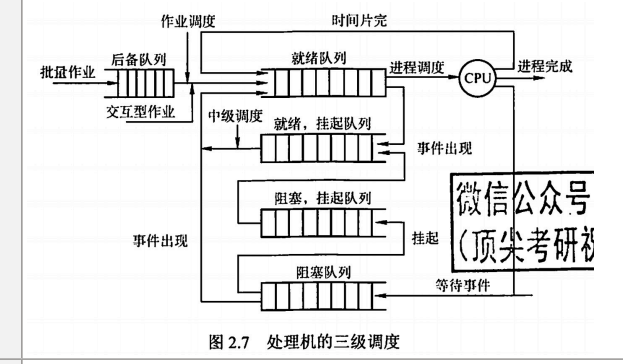


图 2.7 处理机的三级调度

1.高级调度/作业调度	• 是内存与辅存的调度，从后备队列中调度作业 • 每个作业只调入调出一次 • 通常存在于多道批处理系统中
2.中级调度/内存调度	• 目的是提高内存利用率和系统吞吐量 • 将暂时不能运行的进程调到外存等待，设为挂起态，最后修改状态为就绪态，挂在就绪队列上 • 是存储器管理中的对换功能
3.低级调度/进程调度	• 从就绪队列中选取一个进程，调用频率很高 • 各种OS都必须配置这种调度
三种调度的联系	1.作业调度为进程活动做准备，进程调度使进程正常活动 2.中级调度将暂时不能运行的进程挂起，中级调度处于另外两个调度之间 3.调用频率：作业调度<内存调度<进程调度 4.进程调度是最基本的，不可或缺

调度的实现

调度程序的结构图

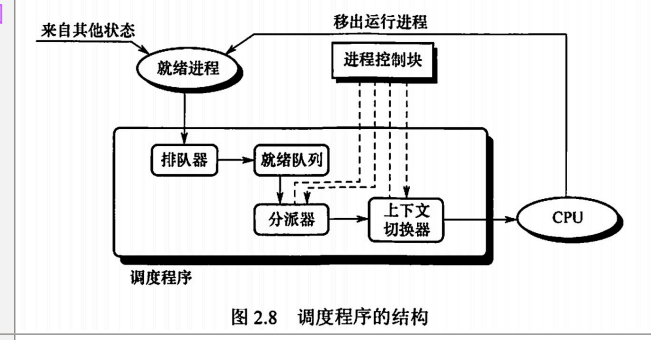


图 2.8 调度程序的结构

调度器的组成

调度器是什么?	用于调度和分派CPU的组件
1.排队器	按策略给就绪进程排出一个或多个队列
2.分派器	从就绪队列中取出进程，并分配CPU
3.上下文切换器	在对处理机进行切换时，会发生两对上下文的切换操作

调度的时机

- 先调度再切换
- 进程从一个状态到另外一个状态变化时，就会触发一次调度
- 调度时机举例
  - 运行的进程运行完毕
  - 运行的进程时间片用完
  - 运行的进程所需资源未准备好
  - 运行的进程自我阻塞
  - 运行的进程出现错误

进程的切换

不能进行调度与切换的情况	• 在处理中断的过程中 • 进程在OS内核临界区中 • 其他需要完全屏蔽中断的原子操作过程中
可以进行调度与切换的情况	• 发生引起调度条件且当前进程无法继续进行下去时（非剥夺调度） • 中断处理结束或自陷处理结束后，被置上请求调度标志（剥夺方式的调度） • 在进程结束时能进行处理机调度 • 创建新进程后能进行处理机调度 • 在系统调用完成并返回用户态时能进行处理机调度 • 进程处于临界区时，只要不破坏临界资源的使用规则，就不影响处理机的调度

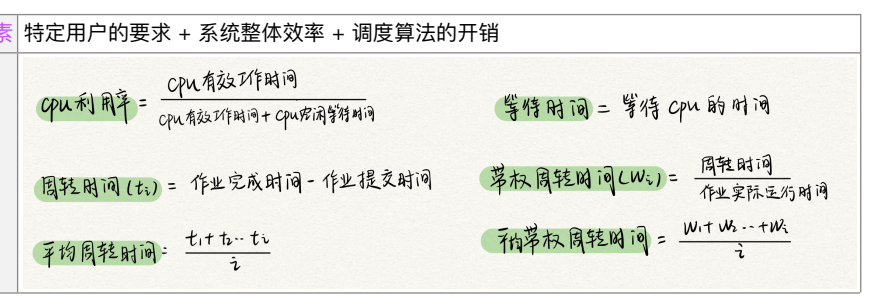
切换的过程

1. 将原进程的信息推入到当前进程的内核堆栈中，并更新堆栈指针
2. 内核从新进程的内核栈中装入新进程的信息
3. 内核更新当前运行的进程空间指针，重设PC寄存器后开始运行新的进程

调度方式

	非抢占调度方式	抢占调度方式
优点	实现简单，系统开销小，适合批处理系统	有利于提高系统吞吐率和响应效率
缺点	不适合分时和大多数的实时系统	必须遵循一定的准则(如优先级，短进程优先，时间片原则)

与调度相关的指标

调度最终目标要考虑的元素	特定用户的要求 + 系统整体效率 + 调度算法的开销
常用的指标	

$$\text{cpu利用率} = \frac{\text{cpu有效工作时间}}{\text{cpu有效工作时间} + \text{cpu空闲等待时间}}$$

$$\text{等待时间} = \text{等待 cpu 的时间}$$

$$\text{周转时间}(t_i) = \text{作业完成时间} - \text{作业提交时间}$$

$$\text{带权周转时间}(W_i) = \frac{\text{周转时间}}{\text{作业实际运行时间}}$$

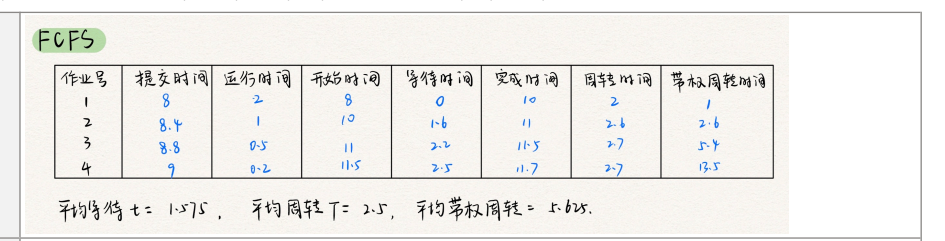
$$\text{平均周转时间} = \frac{t_1+t_2\cdots t_i}{i}$$

$$\text{平均带权周转时间} = \frac{W_1+W_2\cdots+W_i}{i}$$

调度算法【重点！！！】

- 举例：假设有4个作业，提交时间分别为8，8.4，8.8，9，运行时间为2，1，0.5，0.2

FCFS先来先服务

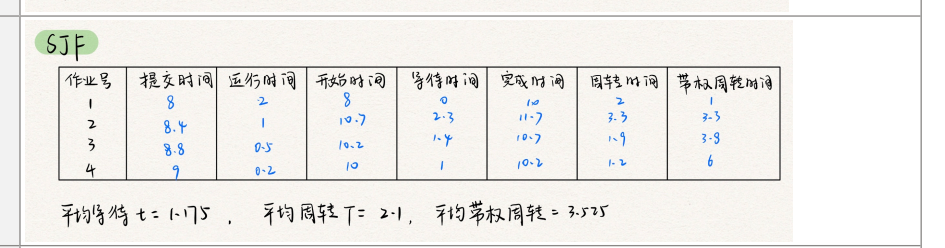


FCFS

作业号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	8	2	8	0	10	2	1
2	8.4	1	10	1.6	11	2.6	2.6
3	8.8	0.5	11	2.2	11.5	2.7	5.4
4	9	0.2	11.5	2.5	11.7	2.7	13.5

平均等待 t = 1.575， 平均周转 T = 2.5， 平均带权周转 = 5.625.

SJF短作业优先

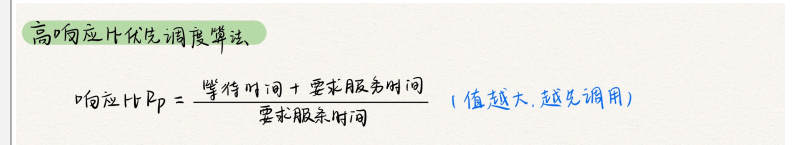


SJF

作业号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	8	2	8	0	10	2	1
2	8.4	1	10.7	2.3	11.7	3.3	3.3
3	8.8	0.5	10.2	1.4	10.7	1.9	3.8
4	9	0.2	10	1	10.2	1.2	6

平均等待 t = 1.175， 平均周转 T = 2.1， 平均带权周转 = 3.525

高响应比优先调度算法



高响应比优先调度算法

$$\text{响应比} R_p = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$ （值越大，越先调用）

其余算法

优先级调度算法
- I/O繁忙作业优先于计算繁忙作业【因为IO操作要及时完成，他没有办法长时间保存数据】
- 系统进程优先于用户进程

时间片轮转算法
- 若时间片过大，退化为FCFS
- 若时间片过小，则切换频繁，处理机开销增大

多级队列算法

多级反馈队列算法需要综合考虑
- 优先级数量，优先级之间的转换规则
- 就绪队列数量，就绪队列的调度算法
- 进程在就绪队列间的迁移条件

调度算法对比图

	FCFS	SJF	高响应比	时间片轮转	多级反馈队列
可抢占?	×	√	√	√	队列内算法不一定
不可抢占?	√	√	√	×	队列内算法一不一定
特点&优点	• 公平 • 实现简单 • 有利于长作业 • 不利于短作业 • 有利于CPU繁忙作业 • 不利于IO繁忙作业	• 平均等待时间最少 • 效率最高	• 兼顾长短作业 • 满足短作业优先且不会发生饥饿现象	• 兼顾长短作业 • 为了多个用户能及时干预系统 • 绝对可抢占的	• 兼顾长短作业 • 有较好的响应时间 • 可行性强
缺点	• 不利于短作业	• 长作业会饥饿 • 估计时间不易确定	• 计算响应比的开销大	• 平均等待时间最长 • 上下文切换浪费时间	无
适用于	无	• 作业调度 • 批处理系统	无	• 分时系统 • 适用于人机交互系统	• 相当通用 • 大家都满意的算法
默认决策模式	非抢占	非抢占	非抢占	抢占	抢占

同步与互斥

选择题常考的点

1、可重入代码/纯代码：一种允许多个进程同时访问的代码；如进程映像中的共享程序段

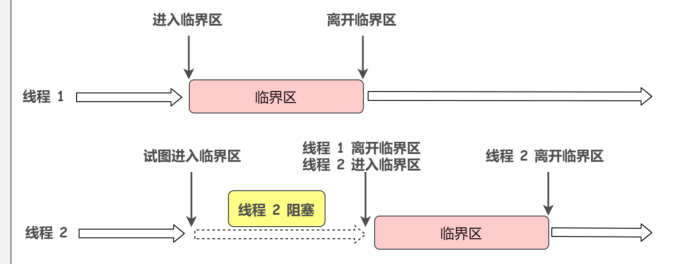
2、PV操作实现的同步的S的初值由用户确定；如果期望的信息还没发送，则对应的初值为0，若信息已存在，则初值为非0的正数
PV操作实现的互斥的S的初值=1

3、执行P操作的进程处于运行态；若P操作之后，S<0，则表示没有可用资源，该进程进入阻塞状态

4、判断代码中的语句是否要互斥执行，可以从以下几个方面考虑【见王道书P121第43题】
- 不同范围的变量不需要互斥（如进程A和进程B都有变量x，这是两个不同范围的变量，不用互斥）
- 对变量赋值前，都有声明语句的话，不需要互斥（如「int a; a=1; int a; a=2」,a=1和a=2不需要互斥）

5、信箱通信是一种间接通信

基本概念

为什么要引入同步互斥?	因为并发进程是异步的，为了协调进程之间的相互制约关系，所以引入同步互斥
进程的异步性是什么?	由于系统的资源有限，进程的执行不是一贯到底的， 而是走走停停，以不可预知的速度向前推进
并发过程执行产生的两种相互制约关系	同步：• 进程 A 应在进程 B 之前执行 互斥：• 进程 A 和进程 B 不能在同一时刻执行
临界资源	一次仅允许一个进程使用的资源；如物理设备(打印机)，共享变量，共享数据，共享缓冲区，公用队列
共享资源	可用被多个进程同时使用的资源；如可重入代码/纯代码，共享程序段，磁盘，非共享数据
临界区	• 访问临界资源的那段代码 • 如果n个进程涉及到了同一个变量A，则A的相关临界区 = 访问临界资源A的那段代码 = n个代码段
示例图	
同步机制遵循的准则 (前3个准则一定要满足)	空闲让进：• 临界区空闲，允许一个进程进入【运行进程访问空闲的临界资源】 忙则等待：• 有进程进入临界区时，其他进程需等待【两个进程不能同时进入临界资源】 有限等待：• 请求访问的进程应保证在有限时间内进入临界区【进程等待进入临界区的时间是有限的】 让权等待：• 进程不能进入临界区时，应该立即释放处理器，防止进程忙等待【不能进入临界区的执行态进程立即放弃CPU】

进程同步与互斥机制1：硬件和软件方法（感觉这里考的不多，主要还是考PV操作）

硬件方法	中断屏蔽方法；硬件指令方法；swap指令；TestAndSet指令（详见书P90） 【硬件方法实现同步时不能实现让权等待】
软件方法	单标志法；双标志法先检查，双标志法后检查；Peterson's Algorithm（详见书P90） 【Peterson算法满足有限等待但不满足让权等待】 【记录型信号量引入阻塞机制，可以让权等待】

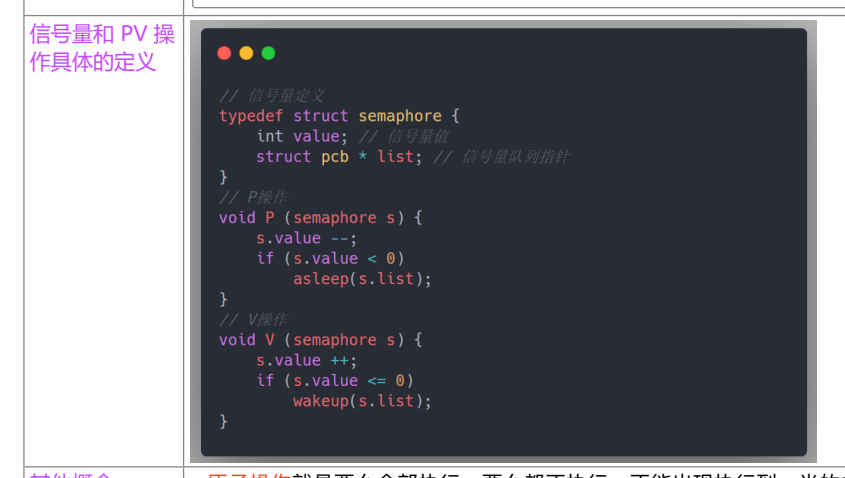
进程同步与互斥机制2：管程

定义	• 管程定义了共享数据结构和各种进程在该数据结构上的全部操作 • 结构类似于Class，把对共享资源的操作封装起来 • 管程支持进程互斥；任何时候只有一个进程在管程中执行 • 管程不仅能实现进程间的互斥，还能实现进程间的同步 • Java 采用管程机制 • Java中管程常见的代码：synchronized关键字，wait()、notify()、notifyAll()
组成	1. 管程的名字 2. 局部于管程内部的共享数据结构或者共享变量说明 3. 对管程内的数据结构进行操作的一组过程 4. 对局部于管程内部的共享数据设置初始值的语句 这类题目如果看到是管程外这个字眼，那就是错误的，管程的组成都是基于管程内部的
管程中设置的条件变量	定义：• 阻塞原因定义为条件变量condition 有两种操作：• x.wait：阻塞进程，将其插入到阻塞队列中 • x.signal：唤醒进程，将其插入到就绪队列中 与信号量的相似点：• wait/signal类似于信号量的P/V操作，实现进程的阻塞/唤醒，但不能说和PV操作相同 与信号量的不同点：• 条件变量没有值，仅实现"排队等待"功能 • 信号量有值，这个值反映了剩余资源数

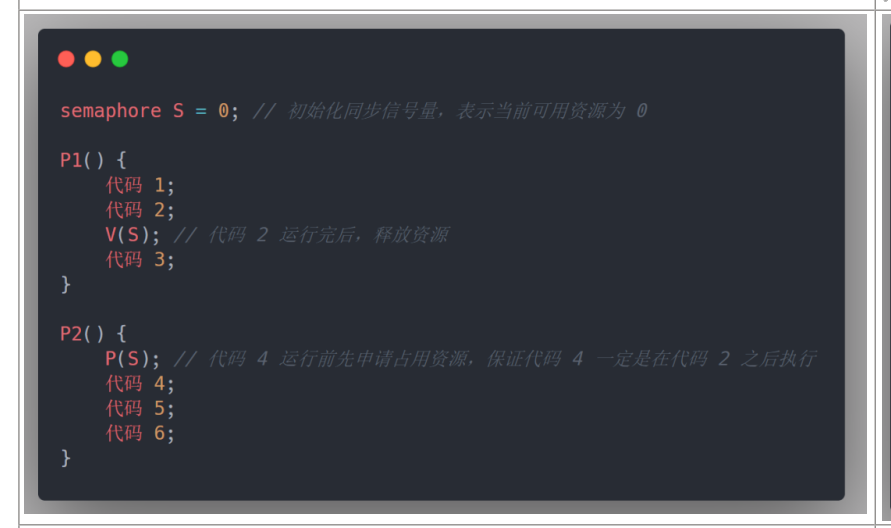
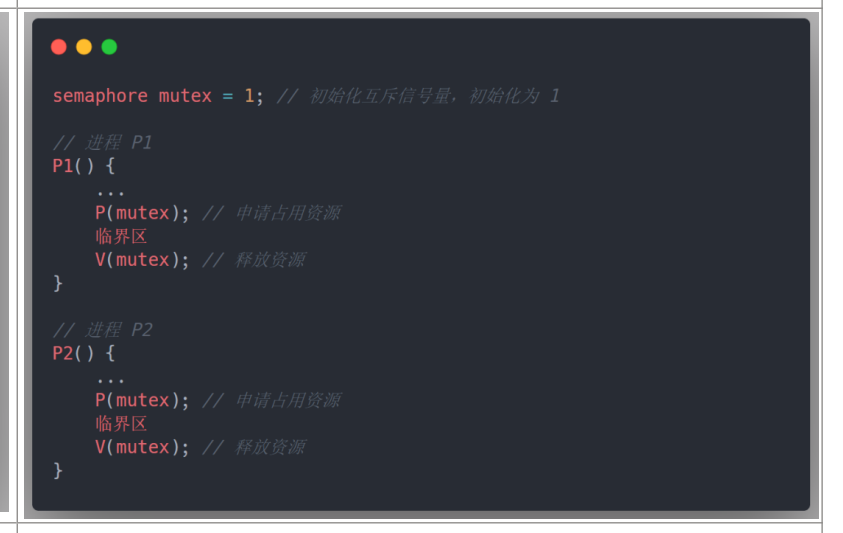
进程同步与互斥机制3：互斥锁

定义	• 解决临界区最简单的工具
特点	• 通常采用硬件机制实现 • 常用于多处理器系统
缺点	• 缺点：忙等待
代码	• acquire()获取 • release()释放

进程同步与互斥机制4：信号量和PC操作

PV操作定义	P操作：• 将信号量值S减 1，表示「申请占用一个资源」 • 如果 s < 0，表示已经没有可用资源，则执行 P 操作的进程被阻塞 • 如果 s ≥ 0，表示现有的资源足够你使用，则执行 P 操作的进程继续执行 • 举例：当信号量的值为2时，表示有 2 个资源可以使用；当信号量的值为-2的时候，表示有两个进程正在等待使用这个资源 V操作：• 将信号量值S加 1，表示「释放一个资源」，即使用完资源后归还资源 • 如果s ≤ 0，表示有某些进程正在等待该资源 • 由于我们已经释放出一个资源了，因此需要唤醒一个等待使用该资源（就绪态）的进程，使之运行下去
常见问题	1、S>0表示有临界资源可供使用，这个时候为什么不需要唤醒进程? • 所谓唤醒进程是从就绪队列（阻塞队列）中唤醒进程，而信号量的值大于 0 表示有临界资源可供使用 • 也就是说这个时候没有进程被阻塞在这个资源上，所以不需要唤醒，正常运行即可 2、S=0表示没有临界资源可供使用，为什么还要唤醒进程? • V 操作是先执行S+1，也就是说，把信号量的值加1后才变成了0 • 在此之前，信号量的值是-1，即有一个进程正在等待这个临界资源，我们需要唤醒它
信号量和 PV 操作具体的定义	
其他概念	• 原子操作就是要么全部执行，要么都不执行，不能出现执行到一半的中间状态的操作 • PV操作是低级进程通信语言
信号量分类	整型信号量：• 该信号量被定义为一个用于表示资源数目的整型量S • 该机制不遵循"让权等待"的准则 记录型信号量：• 是一种不存在"忙等"现象的进程同步机制 • 需要一个用于代表资源数目的变量Value • 需要一个进程链表L，用于链接所有等待该资源的进程 • wait操作 = P操作 = 请求一个资源 • signal操作 = V操作 = 释放一个资源
信号量应用	利用信号量实现同步：• 用资源P操作；释放资源V操作；此时信号量表示资源量 同步信号量初始值不确定，可以设置 信号量最大值 = 最多可以请求的资源数 信号量最小值 = 最大值/初始值 - 最大请求值 利用信号量实现互斥：• PV操作夹着互斥资源；此时信号量表示互斥量 互斥信号量初始值=1，表示临界区只运行一个进程进入，从而实现互斥 互斥信号量=0，表示临界区已经有一个进程进入，临界区外还没有进程等待 互斥信号量<0，表示临界区中有一个进程 |互斥信号量| 表示在临界区外等待进入的进程数 利用信号量实现前驱关系：• 描述程序之间或语句之间的前驱关系

```
// 信号量定义
typedef struct semaphore {
    int value; // 信号量值
    struct pcb * list; // 信号量队列指针
}
// P操作
void P (semaphore s) {
    s.value --;
    if (s.value < 0)
        asleep(s.list);
}
// V操作
void V (semaphore s) {
    s.value ++;
    if (s.value <= 0)
        wakeup(s.list);
}
```

实现进程同步	实现进程互斥
step1：定义一个同步信号量，并初始化为当前可用资源的数量 step2：在优先级较「高」的操作的「后」面执行 V 操作,释放资源 step3：在优先级较「低」的操作的「前」面执行 P 操作,申请占用资源	step1：定义一个互斥信号量，并初始化为 1 step2：把对于临界资源的访问置于 P 操作和 V 操作之间 「P 操作和 V 操作必须成对出现」 缺少 P 操作就不能保证对临界资源的互斥访问 缺少 V 操作就会导致临界资源永远得不到释放、处于等待态的进程永远得不到唤醒
	
对上面↑代码的理解	

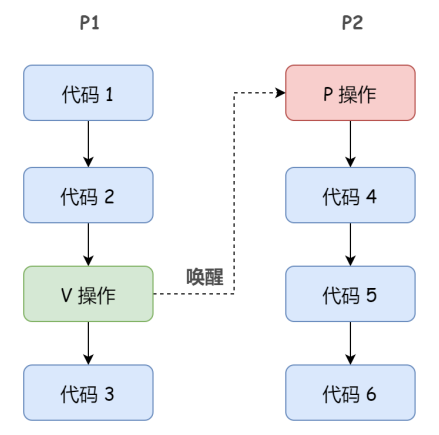
实现进程同步：

```
semaphore S = 0; // 初始化同步信号量，表示当前可用资源为 0

P1() {
    代码 1;
    代码 2;
    V(S); // 代码 2 运行完后，释放资源
    代码 3;
}

P2() {
    P(S); // 代码 4 运行前先申请占用资源，保证代码 4 一定是在代码 2 之后执行
    代码 4;
    代码 5;
    代码 6;
}
```

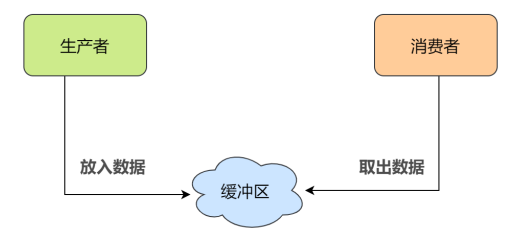
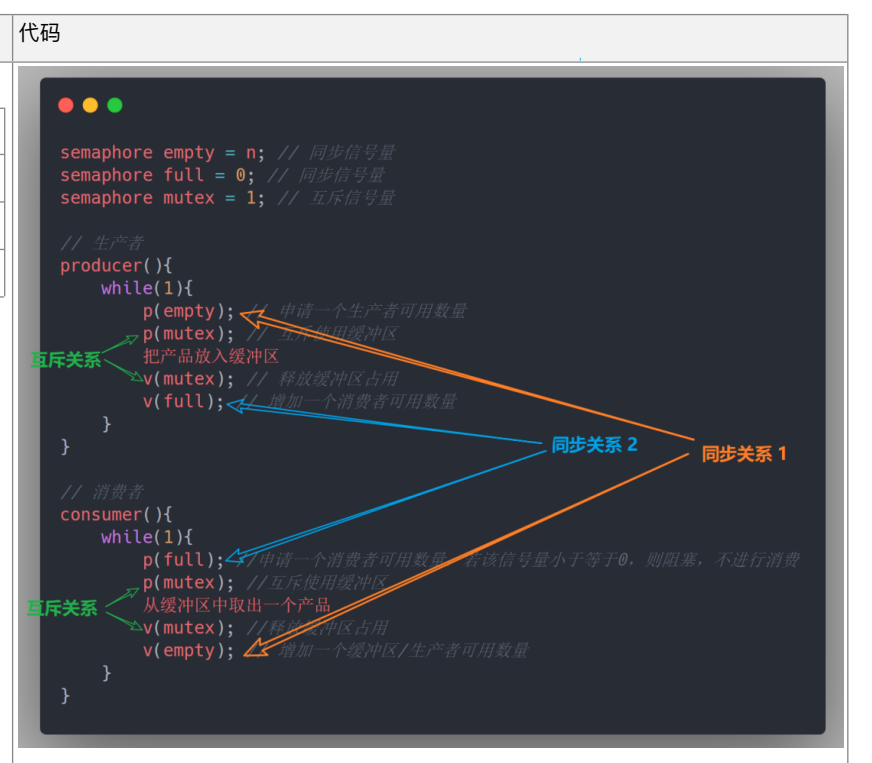
实现进程互斥：

```
semaphore mutex = 1; // 初始化互斥信号量，初始化为 1

// 进程 P1
P1() {
    ...
    P(mutex); // 申请占用资源
    临界区
    V(mutex); // 释放资源
}

// 进程 P2
P2() {
    ...
    P(mutex); // 申请占用资源
    临界区
    V(mutex); // 释放资源
}
```

对上面↑代码的理解



经典同步问题（重中之重！！）

	问题描述	提炼的同步与互斥关系&设置的型号量	代码
生产者-消费者问题	• 系统中有一组生产者进程和一组消费者进程 • 生产者进程每次生产一个产品放入缓冲区 • 消费者进程每次从缓冲区中取出一个产品并使用 • 任何时刻，只能有一个生产者或消费者可以访问缓冲区 	从题目得到的同步互斥关系 缓冲区：生产者、消费者共享一个初始为空、大小为 n 的缓冲区 同步关系 1：只有缓冲区没满时（优先级高），生产者才能把产品放入缓冲区（优先级低），否则必须等待 同步关系 2：只有缓冲区不空时（优先级高），消费者才能从中取出产品（优先级低），否则必须等待 互斥关系：缓冲区是临界资源，各进程必须互斥地访问 设置的信号量 empty：• 同步信号量（对应同步关系 1） • 表示生产者还能生产多少，即还能放入缓冲区多少产品 • 该数量小于等于 0，则生产者不能进行生产 • 初始化为 n full：• 同步信号量（对应同步关系 2） • 表示消费者还能从缓冲区取出多少，即当前缓冲区已有产品的数量 • 该数量小于等于 0，则消费者不能进行读取 • 初始化为 0 mutex：• 互斥信号量 • 实现对缓冲区的互斥访问 • 初始化为 1	
读者-写者问题			
哲学家进餐问题			
吸烟者问题			
参考链接	https://cloud.tencent.com/developer/article/1803377		

```
semaphore empty = n; // 同步信号量
semaphore full = 0; // 同步信号量
semaphore mutex = 1; // 互斥信号量

// 生产者
producer(){
    while(1){
        p(empty); // 申请一个生产者可用数量
        p(mutex); // 互斥使用缓冲区
        把产品放入缓冲区
        v(mutex); // 释放缓冲区占用
        v(full); // 增加一个消费者可用数量
    }
}

// 消费者
consumer(){
    while(1){
        p(full); // 申请一个消费者可用数量，若该信号量小于等于0，则阻塞，不进行消费
        p(mutex); // 互斥使用缓冲区
        从缓冲区中取出一个产品
        v(mutex); // 释放缓冲区占用
        v(empty); // 增加一个缓冲区/生产者可用数量
    }
}
```

死锁

常考点

1.每个进程都分得一个资源，而且还有多余的资源可用让任意一个进程使用，则不出现死锁

2.多道程序技术要求进程间能实现并发，需要实现进程调度保证CPU的工作效率，而并发性的实现需要中断功能的支持

3.系统资源不足不是系统产生死锁的原因，资源不足只会对进程造成"饥饿"【详见王道操作系统书P153第4题】

4.设进程有X个，每个进程最多请求使用3个A资源，系统有11个A资源，则不会出现死锁的最大X为?
答：2*X+1=11, X=5（极端情况时，每个进程分配2个A资源，只要再有一个资源就不会出现死锁）

死锁是什么?

- 死锁是多个进程因为竞争资源而造成的一种互相等待
- 死锁的充分条件：资源分配图中每种资源只有一个，又出现了环路

为什么会出现死锁?

1.系统资源的竞争【空间上】	• 系统中不可剥夺资源不足以满足多个进程 • 只有对不可剥夺资源(如磁带机，打印机)的竞争才可能产生死锁
2.进程推进顺序非法【时间上】	• 进程运行时，请求和释放资源的顺序不当 • 系统对独占资源分配不当

产生死锁的必要条件

- 以下👇四个条件都要满足才会出现死锁

互斥条件	• 多个线程不能同时使用同一个资源
不剥夺条件	• 进程A已经拥有资源1，在自己使用完之前不能被其他进程获取
请求并保持条件	• 进程A已经有资源1，想申请资源2，但是资源2被进程B持有，进程A处于等待状态，但是进程A不释放资源1
循环等待条件	• 两个线程获取资源的顺序构成了环形链

如何解决死锁? 【重点考各种方法，注意不要混淆了这些方法】

死锁预防

	定义	举例
1.破坏互斥条件	• 缺点：如打印机等临界资源只能互斥使用	该方法不太可行
2.破坏不剥夺条件	• 常用于状态易于保存和恢复的资源（CPU的寄存器和内存资源）	剥夺资源法
3.破坏请求并保持条件	• 可能会导致饥饿现象	一次性分配策略
4.破坏循环等待条件	• 可采用顺序资源分配法，但是编号必须相对稳定，限制了新类型设备的增加	资源有序分配策略

死锁避免

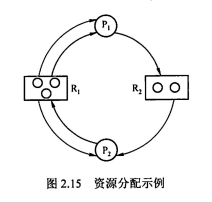
1.安全性算法（具体见下面例题👇）

2.银行家算法（具体见下面例题👇）

死锁包含在不安全状态之中【系统处于安全状态时，一定无死锁；系统处于不安全状态时，不一定出现死锁】

死锁避免时不会限制用户申请资源的顺序；需要进程运行所需资源总量信息；不会给可能导致死锁的进程分配资源

死锁的检测

1.资源分配图	• 资源分配图是一个有向图，用于表示某时刻系统资源与进程之间的状态 • 圆圈代表进程，框代表一类资源；从进程到资源的边叫做请求边；从资源到进程的边叫做分配边  图 2.15 资源分配示例
2.死锁定理	• 死锁定理：S为死锁的条件是当且仅当S状态的资源分配图是不可完全简化的

死锁的解除

1.资源剥夺法	• 挂起某些死锁进程，并抢占它的资源
2.撤销进程法	• 强制撤销部分甚至全部死锁进程并剥夺这些进程的资源
3.进程回退法	• 让一个或多个进程回退到足以回避死锁的地步

各个策略的比较

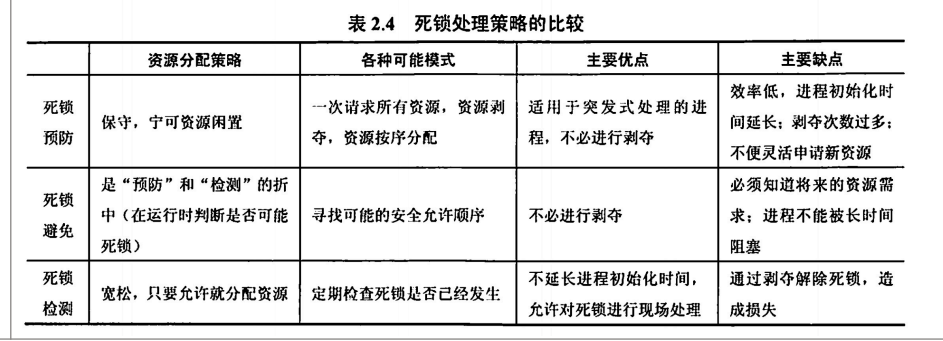
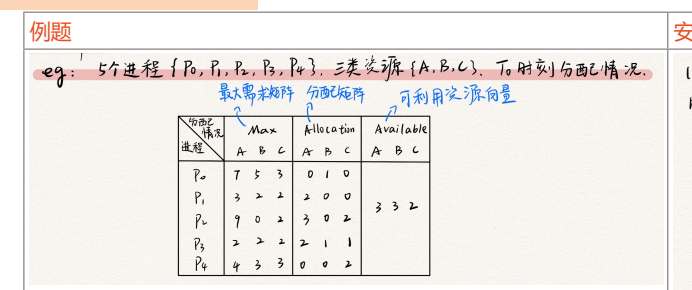


表 2.4 死锁处理策略的比较

	资源分配策略	各种可能模式	主要优点	主要缺点
死锁预防	保守，宁可资源闲置	一次请求所有资源，资源剥夺，资源按序分配	适用于突发式处理的进程，不必进行剥夺	效率低，进程初始化时间延长；剥夺次数过多；不便灵活申请新资源
死锁避免	是"预防"和"检测"的折中（在运行时判断是否可能死锁）	寻找可能的安全允许顺序	不必进行剥夺	必须知道将来的资源需求；进程不能被长时间阻塞
死锁检测	宽松，只要允许就分配资源	定期检查死锁是否已经发生	不延长进程初始化时间，允许对死锁进行现场处理	通过剥夺解除死锁，造成损失

安全性算法和银行家算法举例

例题

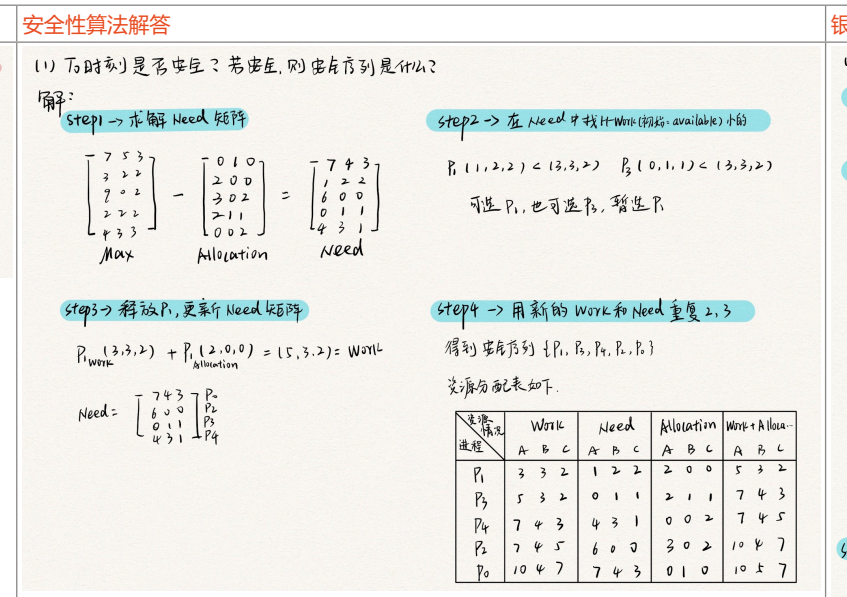


eg: 5个进程{P0, P1, P2, P3, P4}，三类资源{A, B, C}，T0时刻分配情况.

最大需求矩阵　分配矩阵　可利用资源向量

分配情况 进程	Max A B C	Allocation A B C	Available A B C
P0	7 5 3	0 1 0	3 3 2
P1	3 2 2	2 0 0	
P2	9 0 2	3 0 2	
P3	2 2 2	2 1 1	
P4	4 3 3	0 0 2	

安全性算法解答



(1) T0时刻是否安全？若安全，则安全序列是什么？

解：

step1 → 求解 Need 矩阵

$$\begin{bmatrix}7&5&3\\3&2&2\\9&0&2\\2&2&2\\4&3&3\end{bmatrix}_{Max} - \begin{bmatrix}0&1&0\\2&0&0\\3&0&2\\2&1&1\\0&0&2\end{bmatrix}_{Allocation} = \begin{bmatrix}7&4&3\\1&2&2\\6&0&0\\0&1&1\\4&3&1\end{bmatrix}_{Need}$$

step2 → 在 Need 中找比Work(初始：available)小的

P1 (1,2,2) < (3,3,2)　P3 (0,1,1) < (3,3,2)

可选 P1，也可选 P3，暂选 P1

step3 → 释放 P1，更新 Need 矩阵

$P_{1\,Work}(3,3,2) + P_{1\,Allocation}(2,0,0) = (5,3,2) = Work$

$$Need = \begin{bmatrix}7&4&3\\6&0&0\\0&1&1\\4&3&1\end{bmatrix}\begin{matrix}P_0\\P_2\\P_3\\P_4\end{matrix}$$

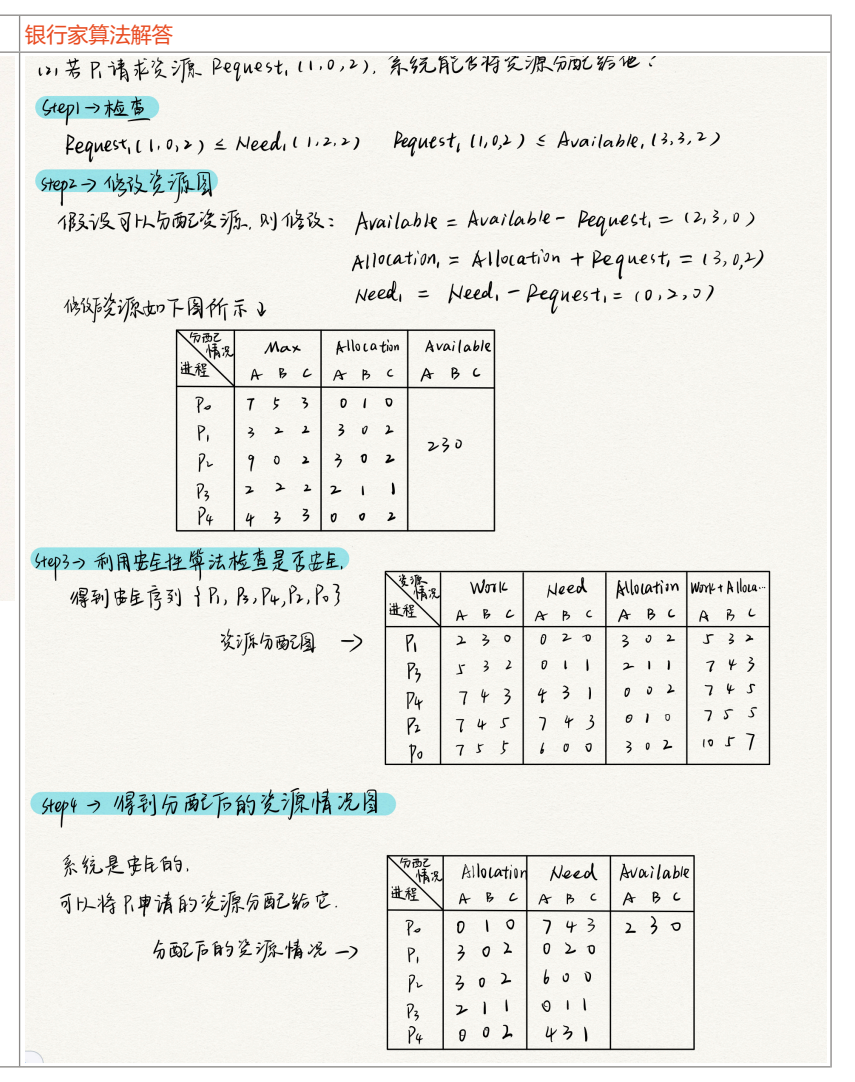
step4 → 用新的 Work 和 Need 重复 2，3

得到安全序列 {P1, P3, P4, P2, P0}

资源分配表如下.

资源情况 进程	Work A B C	Need A B C	Allocation A B C	Work+Allocation A B C
P1	3 3 2	1 2 2	2 0 0	5 3 2
P3	5 3 2	0 1 1	2 1 1	7 4 3
P4	7 4 3	4 3 1	0 0 2	7 4 5
P2	7 4 5	6 0 0	3 0 2	10 4 7
P0	10 4 7	7 4 3	0 1 0	10 5 7

银行家算法解答



(1) 若 P1 请求资源 Request1 (1,0,2)，系统能否将资源分配给他？

Step1 → 检查

$Request_1(1,0,2) \le Need_1(1,2,2)$　$Request_1(1,0,2) \le Available(3,3,2)$

Step2 → 修改资源图

假设可以分配资源，则修改：

$Available = Available - Request_1 = (2,3,0)$

$Allocation_1 = Allocation_1 + Request_1 = (3,0,2)$

$Need_1 = Need_1 - Request_1 = (0,2,0)$

修改资源如下图所示 ↓

分配情况 进程	Max A B C	Allocation A B C	Available A B C
P0	7 5 3	0 1 0	2 3 0
P1	3 2 2	3 0 2	
P2	9 0 2	3 0 2	
P3	2 2 2	2 1 1	
P4	4 3 3	0 0 2	

Step3 → 利用安全性算法检查是否安全.

得到安全序列 {P1, P3, P4, P2, P0}

资源分配图 →

资源情况 进程	Work A B C	Need A B C	Allocation A B C	Work+Alloca... A B C
P1	2 3 0	0 2 0	3 0 2	5 3 2
P3	5 3 2	0 1 1	2 1 1	7 4 3
P4	7 4 3	4 3 1	0 0 2	7 4 5
P2	7 4 5	7 4 3	0 1 0	7 5 5
P0	7 5 5	6 0 0	3 0 2	10 5 7

Step4 → 得到分配后的资源情况图

系统是安全的.

可以将 P1 申请的资源分配给它.

分配后的资源情况 →

分配情况 进程	Allocation A B C	Need A B C	Available A B C
P0	0 1 0	7 4 3	2 3 0
P1	3 0 2	0 2 0	
P2	3 0 2	6 0 0	
P3	2 1 1	0 1 1	
P4	0 0 2	4 3 1	

本章难点和常考点

进程与程序的区别和联系

• 进程是动态的，程序是静态的 • 进程 = 程序+数据+PCB • 程序 = 一组有序的指令集合
• 进程是动态创建消失的，程序是永久存在的，可长期保存 • 进程 = 程序的一次执行过程 • 程序 = 一组代码的集合
• 进程可以执行一个或几个程序，程序也可以构成多个进程
• 进程可以创建进程，程序不可以创建程序

死锁与饥饿【详见左边】

定义	• 死锁 = 两个或多个进程无限地等待一个事件，而该事件只能由这些等待进程之一来产生 • 饥饿/无限期阻塞 = 进程在信号量内无穷等待的情况
差别	• 饥饿并不代表系统一定会死锁，但至少有一个进程的执行被无限期推迟 • 进入饥饿的状态进程可以只有一个，但是进入死锁状态的进程必须大于等于2个 • 处于饥饿状态的基础可以是一个就绪进程，但是死锁状态的进程必定是阻塞进程

同步与互斥的区别和联系【详见左边】

同步	• 进程之间竞争使用临界资源，只能让他们逐个使用
互斥	• 进程之间协同完成任务，在关键点上等待另一个进程发来的消息，以便协同一致